Tomogration — pipeline reference

Every command: what it reads, the exact filenames it looks for, what it writes and where. Generated from the app's own stage definitions.

Notation PositionNNN = a tilt series · <apx> = pixel size to TWO decimals (--angpix 10 → 10.00Apx) · <processing> = warp_tiltseries/ for a trunk run, or jobs/J###/ for a job.

The job model — jobs/J###/

Every run started from the CARD VIEW is a **job** with its own processing directory jobs/J###/ . WarpTools' --input_processing / --output_processing (available on every command) redirect where a run READS its metadata and WRITES its results, so a job reads its parent's directory and writes its own. That is what lets two variants of the same step coexist instead of overwriting one another.

Bulk data (tilt stacks, raw movies) is resolved through the .settings file and is NOT duplicated — a branch only costs the per-series XML (~0.4 MB each). Products that a step genuinely creates (tomograms, subtomograms) live only in the job that made them.

Trunk runs (the classic ▶ Run) skip all this and write to the shared conventional folders above. Both are fine; jobs are what you want when you're comparing variants.

1. DATA PREP

Rename EER + MDOC

```
bash ml_batch_rename_eer_mdoc_mrc_warp_auto.sh
```

Renames Tomo5 beam-shift names (Position_1_2) to Warp form (Position001). Writes listfile_<root>.txt mapping old->new and fixes mdoc dates to yy-mmm-dd (required by Warp).

READS

dirs: .

<source_dir>/*.eer	raw movies from the microscope
<source_dir>/*.mdoc	one per tilt series (SerialEM/Tomo5)

WRITES

dirs: mdocs, frames

<source_dir>/PositionNNN_*.eer	renamed in place
<source_dir>/PositionNNN.mdoc	renamed in place
listfile_Position.txt	map: original Tomo5 name -> PositionNNN

Watch out

- RENAME FIRST, then Sort files. The script globs *.mdoc + matching *.eer together in ONE folder; sorting first breaks that pairing.
- *_override.mdoc files are quarantined to mdocs/bad/ (they would take bogus PositionNNN slots).

IMOD → Warp key

```
python ml_imodtowarpkey_generator_warp_auto.py
```

Builds the IMOD->Warp tilt-number key. mdoc ZValue blocks are in dose-symmetric acquisition order; 3dmod shows tilts in angle order. The key translates between them.

READS

dirs: mdocs

mdocs/PositionNNN.mdoc	one sample mdoc, for the tilt order

WRITES

dirs: .

new_imod_conv_key.txt	IMOD tilt-index -> mdoc-section key

Watch out

- Feeds remake_mdocs, which uses it to translate IMOD tilt numbers.

Inspect tilt stacks

`(no command — an in-app tool)`

Visually review each tilt series (thumbnail grid + 3dmod) and mark bad tilts (IMOD #, ranges OK e.g. 1,4-12,48) or whole series for exclusion before remaking the mdocs.

READS

dirs: Thumbnails, mdocs

`Thumbnails/*.mrc`	per-series montage (one per tilt series)
`mdocs/*.mdoc`	to list the series

WRITES

dirs: mdocs

`exclusion_list.txt`	manual section: `PositionNNN<tab>1,4-12`
`mdocs/bad/`	quarantined whole series (mdoc moved here)
`frames/bad/`	quarantined whole series (.eer moved here)

Watch out

- A BARE `PositionNNN` line (no tilt numbers) = drop the WHOLE series.
- Tilt ranges like `4-12` are expanded before remake_mdocs (its sed can't parse ranges).

Remake MDOCs

`bash ml_batch_remake_mdocs_warp_auto.sh`

Removes excluded tilts from mdocs and renumbers remaining ZValue blocks contiguously, then fixes mdoc date format.

READS

dirs: mdocs

`mdocs/*.mdoc`	the series to rewrite
`exclusion_list.txt`	which tilts/series to drop
`new_imod_conv_key.txt`	IMOD -> mdoc section mapping

WRITES

dirs: mdocs

`mdocs/PositionNNN.mdoc`	rewritten without the excluded tilts

Watch out

- Needs ABSOLUTE paths (it cd's into mdocs/ and reads the lists by name).

2. GAIN

Gain: convert

`module load eman && e2proc3d.py`

Converts a camera .gain reference to .mrc via EMAN2.

READS

dirs: gains

`gains/<name>.gain`	the raw gain reference (name varies per dataset)

WRITES

dirs: gains

`gains/original_gain.mrc`	gain as MRC

Watch out

- The default `gains/original.gain` usually does NOT exist — the real gain file is auto-detected; check the name.

Gain: reciprocal

`module load eman && e2proc2d.py`

Computes the reciprocal gain that Linux WarpTools expects.

READS

dirs: gains

`gains/original_gain.mrc`	from gain_convert

WRITES

dirs: gains

`gains/gain_reciprocal.mrc`	1/gain — this is what create_settings uses

Flags `--process math.reciprocal` on

Watch out

- Linux WarpTools wants the reciprocal; using the plain gain double-applies the correction.

3. FRAMESERIES

Frameseries settings

`WarpTools create_settings`

Writes the Warp frame-series .settings file.

READS

dirs: frames, gains

`frames/*.eer`	just to locate/describe the data
`gains/gain_reciprocal.mrc`	recorded as the gain path

WRITES

dirs: warp_frameseries

`warp_frameseries.settings`	the frameseries settings file

Flags `--folder_data` frames `--folder_processing` warp_frameseries `--output` warp_frameseries.settings `--extension` *.eer `--angpix` 1.57 `--gain_path` gains/gain_reciprocal.mrc `--exposure` 3.5 `--eer_ngroups` 10

Watch out

- Writes NO per-movie data — it only records paths/pixel size/exposure.
- --eer_ngroups IS the correct flag on this Warp 2.0 build.

Motion + CTF

`WarpTools fs_motion_and_ctf`

Per-frame-series motion correction + CTF estimation.

READS

dirs: frames, warp_frameseries

`frames/*.eer`	the raw movies
`warp_frameseries.settings`	paths, angpix, gain, exposure

WRITES

dirs: warp_frameseries

`warp_frameseries/<movie>.xml`	per-movie motion + CTF metadata
`warp_frameseries/average/<movie>.mrc`	aligned average (--out_averages)
`warp_frameseries/averagehalves/`	half-averages (--out_average_halves)

Flags `--settings` warp_frameseries.settings `--m_grid` 1x1x3 `--c_grid` 2x2x1 `--m_range_min` 500 `--m_range_max` 10 `--m_bfac` -500 `--c_range_max` 7 `--c_defocus_max` 8 `--out_averages` on `--out_average_halves` on `--device_list` 0 `--perdevice` 2

Watch out

- MUST pass --out_averages (and --out_average_halves). Without them no aligned averages are written and ts_import fails for EVERY series with 'does not have an aligned average result'.
- Idempotent: re-running skips movies that already have XML.

4. TILT SERIES

Tilt-series settings

`WarpTools create_settings`

Writes the Warp tilt-series .settings file.

READS

dirs: mdocs

`(nothing yet — tomostar/ may be empty)`	

WRITES

dirs: warp_tiltseries

`warp_tiltseries.settings`	the tilt-series settings file

Flags `--folder_data` tomostar `--folder_processing` warp_tiltseries `--output` warp_tiltseries.settings `--extension` *.tomostar `--angpix` 1.57 `--gain_path` gains/gain_reciprocal.mrc `--exposure` 3.5 `--tomo_dimensions` 4096x4096x3088

Watch out

- Run BEFORE ts_import. Seeing '0 files found / tomostar not found' here is EXPECTED — tomostar/ doesn't exist yet.

Import tilt series

`WarpTools ts_import`

Builds .tomostar files by pairing mdocs with frameseries.

READS

dirs: mdocs, warp_frameseries

`mdocs/*.mdoc`	tilt angles, order, exposure
`warp_frameseries/<movie>.xml`	the per-movie results + aligned averages

WRITES

dirs: tomostar, warp_tiltseries

`tomostar/PositionNNN.tomostar`	one per tilt series — the item list
`warp_tiltseries/PositionNNN.xml`	per-series metadata (created here)

Flags `--mdocs` mdocs `--frameseries` warp_frameseries `--tilt_exposure` 3.5 `--min_intensity` 0 `--dont_invert` on `--output` tomostar `--override_axis`

Watch out

- CANNOT be scoped with --input_data (only --mdocs / --pattern), which is why subset work uses a separate project folder.

Build tilt stacks

`WarpTools ts_stack`

Builds aligned tilt stacks (.st) per tilt series for AreTomo.

READS

dirs: warp_tiltseries, tomostar

`tomostar/*.tomostar`	the series
`warp_frameseries/average/*.mrc`	the aligned averages

WRITES

dirs: warp_tiltseries/tiltstack

`warp_tiltseries/tiltstack/PositionNNN/PositionNNN.st`	the tilt stack
`warp_tiltseries/tiltstack/PositionNNN/*.rawtlt`	tilt angles

Flags `--settings` warp_tiltseries.settings `--angpix` `--device_list` 0 `--perdevice` 2

Watch out

- Run ts_import first; a missing mdoc entry stalls a stack.

5. ALIGNMENT

Align (AreTomo2)

`bash`

Marker-free tilt-series alignment (+optional recon). Writes .xf/.tlt per series under <output>/Imod/.

READS

dirs: warp_tiltseries/tiltstack

`warp_tiltseries/tiltstack/*/*.st`	the tilt stacks

WRITES

dirs: aretomo_output

`aretomo_output[-vN]/PositionNNN/`	per-series AreTomo output
`aretomo_output[-vN]/Imod/PositionNNN/PositionNNN.xf`	the ALIGNMENT — this is the only thing Warp needs
`aretomo_output[-vN]/PARAMETERS.txt`	what was run

Flags `ARETOMO_GPUS` 0 1 2 3 `ARETOMO_JOBS_PER_GPU` 1 `ARETOMO_ALIGNZ` 670 `ARETOMO_VOLZ` 3088 `ARETOMO_OUTBIN` 8 `ARETOMO_DARKTOL` 0.000001 `ARETOMO_TILTCOR` 0 `ARETOMO_FLIPVOLZ` 1 `ARETOMO_WBP` 1 `ARETOMO_TILTAXIS` `ARETOMO_PATCH` `ARETOMO_ALIGN` 1 `ARETOMO_BIN` /ceph/groups/structbio/Programs/AreTomo2/AreTomo2 `ARETOMO_CUDA_LIB`

Watch out

- Auto-VERSIONS: each run makes a NEW aretomo_output-vN folder. A killed run leaves a PARTIAL folder — a later version can have FEWER .xf than an earlier one. Check: for d in aretomo_output*; do echo "$d: $(find $d -name '*.xf'|wc -l)"; done
- Set ARETOMO_VOLZ=0 for ALIGNMENT-ONLY (skips the slow tomogram; you only need the .xf).

Refine alignment (train)

`bash`

Deep-learning REFINEMENT of an EXISTING alignment by TRAINING a model on this set (warpem/miss-alignment). Does NOT align raw stacks — the docs state 'miss-alignment starts from an initially coarse aligned dataset', so coarse-align FIRST (AreTomo → ts_import_alignments).

READS

dirs: warp_tiltseries, warp_tiltseries/tiltstack

`warp_tiltseries/PositionNNN.xml`	reads AND REWRITES these in place
`warp_tiltseries/tiltstack/`	the stacks

WRITES

dirs: warp_tiltseries

`warp_tiltseries/PositionNNN.xml`	refined alignment written back in place
`warp_tiltseries/models/`	the trained model checkpoints

Flags `MA_TRAINING_DEVICES` 0 `MA_RECON_DEVICES` 0,0,0 `MA_DATALOADERS` 5 `MA_POOL_SIZE` 2000 `MA_START_ITER` `MA_PREPARE_STACKS` 10.0 `MA_CONDA_ENV` miss-alignment

Watch out

- REFINES an existing coarse alignment — it CANNOT align raw stacks. Run AreTomo -> ts_import_alignments -> select FIRST, or you get a featureless tomogram.
- MA_TRAINING_DEVICES must be a SINGLE GPU. Speed comes from MA_RECON_DEVICES.

Refine alignment (infer)

`bash`

Reuse a model trained by 'miss-alignment (train)' to align THIS (usually larger) dataset with NO retraining — runs 'miss-alignment infer', loading iterN/model.ckpt for each iteration.

READS

dirs: warp_tiltseries, warp_tiltseries/tiltstack

`<MA_MODEL_RUN_DIR>/iterN/model.ckpt`	models from a finished training run
`warp_tiltseries/PositionNNN.xml`	reads AND rewrites in place

WRITES

dirs: warp_tiltseries

`warp_tiltseries/PositionNNN.xml`	refined alignment, no training

Flags `MA_MODEL_RUN_DIR` `MA_INFER_DEVICES` 0,1,2,3 `MA_START_ITER` `MA_PREPARE_STACKS` 10.0 `MA_CONDA_ENV` miss-alignment

Watch out

- The larger set still needs its OWN coarse alignment imported first.

Import alignments

`WarpTools ts_import_alignments`

Imports AreTomo .xf/.tlt alignments back into Warp.

READS

dirs: aretomo_output, warp_tiltseries

`aretomo_output[-vN]/Imod/PositionNNN/PositionNNN.xf`	AreTomo's alignment
`warp_tiltseries/PositionNNN.xml`	the series to update

WRITES

dirs: warp_tiltseries

`warp_tiltseries/PositionNNN.xml`	tilt axis + shifts written INTO the xml

Flags `--settings` warp_tiltseries.settings `--alignments` aretomo_output/Imod/ `--alignment_angpix` 1.57

Watch out

- Points at the NEWEST aretomo_output*/Imod/ by default. If that run was killed it may be INCOMPLETE -> 'Could not find PositionNNN.xf' for most series. Type a specific version (e.g. aretomo_output-v2/Imod/) to override.
- A FAILED import DESELECTS every failed series. Re-enable with sync_selection -> Select.

Sync selection

`WarpTools change_selection`

Deselects tilt series with no AreTomo alignment so ts_reconstruct won't crash trying to reconstruct them.

READS

dirs: warp_tiltseries

`warp_tiltseries/PositionNNN.xml`	the selection flag lives here

WRITES

dirs: warp_tiltseries

`warp_tiltseries/PositionNNN.xml`	sets selected/deselected in place

Flags `--settings` warp_tiltseries.settings `--input_data`

Watch out

- ANY failed per-item run DESELECTS that item, and downstream steps then SILENTLY SKIP it. After any partial failure, run this with --select.
- WarpTools needs EXACTLY ONE mode flag (--select / --deselect / ...).

6. CTF

Defocus handedness

`WarpTools ts_defocus_hand`

Checks (and optionally flips) defocus handedness. Exactly one mode runs per invocation.

READS

dirs: warp_tiltseries

`warp_tiltseries/PositionNNN.xml`	CTF fits

WRITES

dirs: warp_tiltseries

`warp_tiltseries/PositionNNN.xml`	sets the defocus handedness

Flags `--settings` warp_tiltseries.settings

Watch out

- Run --check first; only then --set_*. Exactly ONE mode flag.

CTF estimation

`WarpTools ts_ctf`

Per-tilt CTF refinement across each series.

READS

dirs: warp_tiltseries, warp_tiltseries/tiltstack

`warp_tiltseries/PositionNNN.xml`	series metadata
`warp_tiltseries/tiltstack/*/*.st`	the stacks to fit

WRITES

dirs: warp_tiltseries

`warp_tiltseries/PositionNNN.xml`	CTF written INTO the xml: <CTF><Param Name="Defocus"> (per-series mean, um), DefocusDelta/DefocusAngle (astigmatism), and <GridCTF> (per-TILT defocus, one Node per tilt)

Flags `--settings` warp_tiltseries.settings `--range_high` 7 `--defocus_max` 8 `--device_list` 0 `--perdevice` 2

Watch out

- No new files — it EDITS the per-series xml in place.

7. RECONSTRUCT

Tomogram reconstruction

WarpTools ts_reconstruct

Back-projects aligned, CTF-corrected tilts into 3D tomograms.

READS

dirs: warp_tiltseries

warp_tiltseries/PositionNNN.xml	alignment + CTF
warp_tiltseries/tiltstack/*/*.st	the stacks

WRITES

dirs: warp_tiltseries/reconstruction

warp_tiltseries/reconstruction/PositionNNN_<apx>Apx.mrc	the full tomogram, e.g. Position042_12.56Apx.mrc

Flags --settings warp_tiltseries.settings --angpix 10 --input_data --output_processing --device_list 0 --perdevice 1 --deconv --deconv_strength --deconv_falloff --deconv_highpass --halfmap_frames --halfmap_tilts --dont_overwrite --subvolume_size --subvolume_padding --keep_full_voxels --dont_invert --dont_normalize --dont_mask

Watch out

- is the pixel size to TWO DECIMALS: --angpix 10 -> '10.00Apx'.
- Leaving --angpix blank means NATIVE (1.57 A) -> ~260x the volume, tens of GB and ~40 min EACH. Use ~10 A for viewing/picking.
- Needs WARP_FORCE_MRC_FLOAT32=1 for IMOD/3dmod to read the output.

8. PICK

Template matching

WarpTools ts_template_match

CTF-aware template matching to locate particles (apoferritin example values — adapt per target).

READS

dirs: warp_tiltseries/reconstruction

warp_tiltseries/reconstruction/PositionNNN_<apx>Apx.mrc	the FULL TOMOGRAM at --tomo_angpix. It REUSES this; it does not rebuild it.
warp_tiltseries/template/emd_<code>.mrc	the template (auto-downloaded by --template_emdb) or your --template_path
warp_tiltseries/PositionNNN.xml	alignment + CTF

WRITES

dirs: warp_tiltseries/matching

warp_tiltseries/matching/PositionNNN_<apx>Apx<SUFFIX>.star	THE PEAK LIST. <SUFFIX> = --override_suffix verbatim if set, else the template-derived name (_emd_70905 / _<template stem>)
warp_tiltseries/matching/PositionNNN_<apx>Apx<TEMPLATE>_corr.mrc	correlation volume — named after the TEMPLATE, NOT your override_suffix
warp_tiltseries/matching/PositionNNN_<apx>Apx<TEMPLATE>_angleid.mrc	best-orientation map (same template-based naming)
warp_tiltseries/template/emd_<code>.mrc	the downloaded EMDB map

Flags --settings warp_tiltseries.settings --tomo_angpix 10 --template_emdb --template_path --override_suffix --subdivisions 3 --template_diameter --symmetry C1 --whiten on --optimize_poses --check_hand 2 --npeaks 2000 --peak_distance --max_missing_tilts 2 --subvolume_size 192 --device_list --perdevice 1

Watch out

- --tomo_angpix MUST equal an --angpix you ALREADY ran ts_reconstruct at, or every series fails with 'A reconstruction at the desired resolution was not found.'
- --override_suffix renames ONLY the .star. The _corr.mrc / _angleid.mrc keep the TEMPLATE name — so a viewer needs -mp '*.star' but -cvp '*